



AMPLIVO

Backend / API & Technology Stack Documentation

Complete technical reference for the Amplivo Digital Marketing ERP & Client Portal
— covering architecture, API endpoints, security model, data models, and
deployment strategy.

Table of Contents

1.	Technology Stack Overview	
2.	System Architecture	
3.	Backend Core & Configuration	
4.	Database & ORM	
5.	Middleware Stack	
6.	Authentication API Reference	
7.	Business Module APIs	
8.	Security Model	
9.	Core Data Models	
10.	Frontend Technology	
11.	Environment Variables Reference	
12.	Deployment & Infrastructure	
13.	Health & Monitoring	



SECTION 1

Technology Stack Overview

Backend Stack

Runtime & Language

Python 3.12+

AsyncIO

uvicorn 0.32

Async-first Python on Uvicorn ASGI server with standard extras (websockets, lifespan).

Web Framework

FastAPI 0.115

Starlette

Pydantic v2

FastAPI with automatic OpenAPI schema generation, dependency injection, Pydantic v2 validation.

Database ORM

SQLAlchemy 2.0

asyncpg 0.30

Alembic 1.13

SQLAlchemy 2.0 async ORM with asyncpg driver and Alembic for schema migrations.

Auth & Security

PyJWT 2.9

passlib 1.7.4

bcrypt 4.0.1

JWT-based stateless authentication with bcrypt password hashing (12 rounds).

Data Validation

Pydantic 2.9.2

pydantic-settings 2.5

email-validator

Pydantic v2 for request/response validation and settings management via environment variables.

Cloud Infrastructure

Supabase PostgreSQL

Render

Vercel

Supabase managed PostgreSQL with Supervisor pooler. Backend on Render, frontend on Vercel.

Frontend Stack

Framework

Next.js 16

React 19

TypeScript 5.9

Next.js App Router with React Server Components and TypeScript strict mode.

Styling

TailwindCSS v4

clsx

tailwind-merge

Utility-first CSS with Tailwind v4, merge utilities for conditional class management.

API Client

Axios 1.18

JWT Bearer Auth

Auto-refresh

State & Forms

Zustand 5

React Hook Form 7

Zod 4

Zustand for global state, React Hook Form + Zod for type-safe form validation.

Axios with interceptors for automatic token attachment and silent token refresh on 401.

Charts & Icons

Recharts 3.9

Lucide React

React Icons 5

Recharts for dashboard analytics, Lucide + React Icons for UI iconography.

Dev Tooling

ESLint 9

PostCSS

Node 20+

ESLint with Next.js config, PostCSS pipeline for Tailwind, Node 20 LTS runtime.



SECTION 2

System Architecture

Amplivo follows a **monorepo two-tier architecture**: a Next.js frontend and FastAPI backend deployed independently, communicating via RESTful JSON API.

Deployment Architecture



End Users / Browsers

Internal staff, marketing agents, clients via Client Portal — HTTPS only



Vercel CDN (Frontend)

Next.js 16 · amplivo-front-and-backend.vercel.app · Edge network, SSR, static assets

↕ HTTPS REST API



Render Web Service (Backend)

FastAPI 0.115 · amplivo-front-and-backend.onrender.com · Python 3.12 · uvicorn

↕ SSL PostgreSQL (asyncpg)



Supabase PostgreSQL

Managed Postgres · Supavisor pooler (port 6543, transaction mode) · ap-south-1 Mumbai

Backend Layered Architecture

```
Backend/app/
├─ api/v1/           ← HTTP route handlers (thin layer)
│  └─ auth/          ← Authentication endpoints
│     └─ router.py    ← Central API router (35 modules)
├─ modules/          ← Feature domain modules (35+)
│  └─ crm/            ← CRM: clients, contacts, proposals
│  └─ leads/          ← Lead management and sales pipeline
│  └─ finance/        ← Invoices, payments, expenses
│  └─ ...             ← 30+ other business modules
├─ services/         ← Business logic layer
├─ repositories/     ← Data access layer
├─ models/           ← SQLAlchemy ORM models
├─ schemas/          ← Pydantic request/response schemas
├─ middleware/       ← 8 middleware components
├─ core/             ← Config, exceptions, pagination
├─ db/               ← Database session management
├─ dependencies/     ← FastAPI DI providers (auth, db, rbac)
└─ utils/            ← JWT, password, request context
```

35 Domain Modules

- Activity Timeline
- Analytics
- Approval System
- Campaigns
- Careers
- Case Studies
- Client Portal
- CMS
- Companies
- Consultation Requests
- Contact Forms
- Content Calendar
- Creative
- CRM
- FAQs
- File Manager
- Finance
- Influencers
- Leads
- Marketing Automation
- Meetings
- Messaging
- Notifications
- Paid Ads
- Portfolio
- SEO
- Settings
- Social
- Support Tickets
- Tasks
- Testimonials
- Timesheets
- Users
- Websites



SECTION 3

Backend Core & Configuration

Application Bootstrap (app/main.py)

- **Title:** Amplivo ERP Auth Service | **Version:** 2.0.0
- **API Prefix:** `/api/v1` | **Swagger UI:** `/api/v1/docs` | **ReDoc:** `/api/v1/redoc`
- **OpenAPI JSON:** `/api/v1/openapi.json`
- **Static Files:** `/uploads` → `Backend/uploads/`
- **Lifespan:** Validates DB connection at startup, disposes engine pool on shutdown
- **Root** `/` : Redirects to Swagger UI

Settings (app/core/config.py)

All settings managed via `pydantic-settings` with env var overrides. Singleton via `@lru_cache` .

SETTING	DEFAULT	DESCRIPTION
<code>PROJECT_NAME</code>	Amplivo ERP Auth Service	Application name in docs
<code>API_V1_PREFIX</code>	<code>/api/v1</code>	All API routes prefixed with this
<code>ENVIRONMENT</code>	development	Runtime environment flag
<code>DATABASE_URL</code>	postgresql+asyncpg://...	Async PostgreSQL connection string
<code>DB_POOL_SIZE</code>	5	SQLAlchemy connection pool size
<code>DB_MAX_OVERFLOW</code>	10	Max connections above pool size
<code>DB_POOL_RECYCLE_SECONDS</code>	1800	Connection recycle interval (30 min)
<code>DB_SSL_MODE</code>	require	"require" for remote; "disable" for local
<code>JWT_SECRET_KEY</code>	CHANGE_ME	HMAC secret for JWT signing
<code>JWT_ALGORITHM</code>	HS256	JWT signing algorithm
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	30	Access token TTL
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	7	Refresh token TTL
<code>BCRYPT_ROUNDS</code>	12	Password hashing cost factor
<code>MAX_FAILED_LOGIN_ATTEMPTS</code>	5	Lockout threshold
<code>ACCOUNT_LOCK_MINUTES</code>	15	Lockout duration
<code>RATE_LIMIT_LOGIN_PER_MINUTE</code>	5	Login rate limit per IP
<code>RATE_LIMIT_REGISTER_PER_MINUTE</code>	3	Register rate limit per IP
<code>RATE_LIMIT_REFRESH_PER_MINUTE</code>	10	Token refresh rate limit

SETTING	DEFAULT	DESCRIPTION
SESSION_INACTIVITY_TIMEOUT_MINUTES	60	Session inactivity timeout



SECTION 4

Database & ORM

The application uses **PostgreSQL** hosted on **Supabase** with the **Supavisor** connection pooler (port 6543, transaction mode). Driver: **asyncpg**.

⚠ Supavisor & Prepared Statements: Because Supavisor reuses physical connections across logical sessions, prepared statement names must be globally unique. The app uses `prepared_statement_name_func = lambda: f"__asyncpg_{uuid.uuid4().hex}__"` and disables asyncpg's statement cache (`statement_cache_size=0`) to prevent `DuplicatePreparedStatementError` .

Connection Pool (Production)

PARAMETER	VALUE	PURPOSE
pool_size	3	Base connection count
max_overflow	5	Max burst connections
pool_recycle	300s	Recycle idle connections
pool_timeout	60s	Acquire timeout
pool_pre_ping	True	Test connections before use
DB_STARTUP_TIMEOUT	10s	Boot connectivity check

ORM Pattern

- SQLAlchemy 2.0 with `async_sessionmaker` and `AsyncSession`
- Sessions use `expire_on_commit=False` , `autoflush=False`
- Sessions injected via FastAPI dependency `get_db()`
- Alembic manages all schema migrations — applied at each deploy
- All primary keys: `UUID v4` (Python-generated)
- Timestamps: `DateTime(timezone=True)` with `func.now()` server defaults
- Soft deletes via `is_deleted / deleted_at / deleted_by` on User model

Core Database Tables

TABLE	PURPOSE
<code>users</code>	Central identity: auth + RBAC + org-structure
<code>refresh_tokens</code>	Hashed refresh tokens with device/IP metadata
<code>user_sessions</code>	Device-level session tracking for multi-device management
<code>audit_logs</code>	Immutable event log with JSONB old/new data snapshots
<code>email_verification_tokens</code>	One-time tokens for email verification (24h TTL)

TABLE	PURPOSE
password_reset_tokens	Time-limited tokens for password reset (30 min TTL)
roles	RBAC role definitions
departments	Organizational departments
branches	Company branch/office locations
clients	CRM client companies
leads	Sales lead records
invoices	Finance invoices
+ 30 module tables	All business module domain data



SECTION 5

Middleware Stack

Starlette processes middleware LIFO (last-added = outermost). Order below shows **request-processing order** (outermost → innermost):

CORSMiddleware

1

Handles preflight OPTIONS before any auth checks. Allows: localhost:3000/3001, Vercel deployments. Credentials allowed, all methods/headers.

UnhandledErrorMiddleware

2

Catches unhandled exceptions and converts to structured JSON — ensures CORS headers preserved even on uncaught DB errors.

RateLimiterMiddleware

3

Fixed-window in-memory limiter by (path, IP). Limits: login 5/min, register 3/min, refresh 10/min. Returns HTTP 429.

SecurityHeadersMiddleware

4

Injects X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy, Permissions-Policy, CSP. Docs paths get relaxed CSP.

AuditMiddleware

5

Parses User-Agent once per request; caches IP/device/browser/OS on `request.state.client_context` for service-layer audit logging.

SessionMiddleware

6

Resolves `request.state.session_id` from authenticated user's active session for session-scoped operations.

ActivityMiddleware

7

Updates last_seen timestamp and session last_activity on authenticated requests for inactivity tracking.

AuthenticationMiddleware

8

Decodes Bearer JWT → sets `request.state.user_id`. Never rejects — route-level `get_current_user` dependency enforces protection.



SECTION 6

Authentication API Reference

Base URL: <https://amplivo-front-and-backend.onrender.com/api/v1>

Core Authentication Endpoints

METHOD	PATH	AUTH	DESCRIPTION
POST	/auth/register	Public	Register new user → returns UserRead (201). Rate-limited 3/min.
POST	/auth/login	Public	Authenticate with email/username + password → JWT pair. Rate-limited 5/min.
POST	/auth/refresh	Public	Exchange refresh token → new access+refresh pair. Rate-limited 10/min.
POST	/auth/logout	Public	Revoke refresh token (ends session). Returns success message.
GET	/auth/me	JWT	Get authenticated user profile with role → UserRead.
POST	/auth/change-password	JWT	Change password (requires current + new password).
GET	/auth/check-email	Public	Check if email is registered. Query: ?email=
GET	/auth/check-username	Public	Check if username is taken. Query: ?username=

Email Verification Endpoints

METHOD	PATH	AUTH	DESCRIPTION
POST	/auth/send-verification	JWT	Send/resend verification email to current user
POST	/auth/verify-email	Public	Verify email using one-time token → updated UserRead
POST	/auth/resend-verification	Public	Request verification by address without auth (safe: no info leak)
GET	/auth/verification-status	JWT	Get email verification status and timestamp

Password Reset Endpoints

METHOD	PATH	DESCRIPTION
POST	/auth/forgot-password	Request reset email. Safe response regardless of email existence.
POST	/auth/reset-password	Reset password using valid token. Invalidates all existing sessions.

Session Management Endpoints

METHOD	PATH	DESCRIPTION
GET	/auth/sessions	List current user's active sessions (device, browser, OS, IP, last activity)
GET	/auth/devices	List all devices user ever logged in from (historical)
DELETE	/auth/sessions/current	Terminate the current active session
DELETE	/auth/sessions/{session_id}	Terminate a specific session by UUID
DELETE	/auth/sessions	Terminate all sessions except current (logout all other devices)

Token Response Schema

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
  "token_type": "bearer"
}
```



SECTION 7

Business Module APIs

All business module endpoints require **Bearer JWT** via `Authorization: Bearer <token>`. List endpoints support pagination: `?page=1&page_size=20&search=&sort_by=&sort_order=asc`. Pagination response: `{"items":[], "total":N, "page":1, "page_size":20, "pages":P}`

CRM — Clients (/api/v1/clients)

METHOD	PATH	ROLE	DESCRIPTION
GET	/clients	any	Paginated list; filter by status, type, assigned_to, branch, is_active
POST	/clients	sales	Create new client
GET	/clients/me	client	Get own company (client portal users)
GET	/clients/{id}	any	Get single client (tenant-scoped for portal users)
PUT	/clients/{id}	sales	Update client
DELETE	/clients/{id}	sales	Delete client (204)
GET	/clients/{id}/contacts	any	List client contacts
POST	/clients/{id}/contacts	sales	Add contact to client
PUT	/contacts/{id}	sales	Update contact
DELETE	/contacts/{id}	sales	Delete contact
GET	/clients/{id}/addresses	any	List client addresses
POST	/clients/{id}/addresses	sales	Add address
GET	/clients/{id}/documents	any	List client documents
POST	/clients/{id}/documents	sales	Upload document
GET	/clients/{id}/notes	any	List client notes
POST	/clients/{id}/notes	sales	Add note
GET	/clients/{id}/proposals	any	List proposals
POST	/clients/{id}/proposals	sales	Create proposal
PUT	/proposals/{id}	sales	Update proposal
DELETE	/proposals/{id}	sales	Delete proposal

CRM — Leads (/api/v1/leads)

METHOD	PATH	DESCRIPTION
GET	/leads	Paginated leads; filter by status, priority, source, assigned_to, client
POST	/leads	Create lead
GET	/leads/dashboard-stats	Live sales KPI rollup (role-scoped)
GET	/leads/sales-reports	Sales reports: conversion, revenue, won-vs-lost. Query: ?type=
GET	/leads/{id}	Get single lead (tenant-scoped)
PUT	/leads/{id}	Update lead
DELETE	/leads/{id}	Delete lead
POST	/leads/{id}/convert	Convert lead to client
GET	/leads/{id}/activities	List lead activities
POST	/leads/{id}/activities	Add activity to lead
GET	/leads/{id}/followups	List follow-ups
POST	/leads/{id}/followups	Add follow-up
GET	/lead-sources	List lead sources
POST	/lead-sources	Create lead source
GET	/sales-pipeline	List pipeline stages
POST	/sales-pipeline	Create pipeline stage
PUT	/sales-pipeline/{id}	Update pipeline stage
DELETE	/sales-pipeline/{id}	Delete pipeline stage

Finance (/api/v1/finance)

METHOD	PATH	DESCRIPTION
GET	/finance/invoices	List invoices; filter by client_id, status
POST	/finance/invoices	Create invoice
GET	/finance/invoices/{id}	Get invoice (tenant-scoped)
PUT	/finance/invoices/{id}	Update invoice
DELETE	/finance/invoices/{id}	Delete invoice
GET	/finance/invoices/{id}/items	List invoice line items

METHOD	PATH	DESCRIPTION
POST	/finance/invoices/{id}/items	Add invoice line item
PUT	/finance/invoice-items/{id}	Update line item
DELETE	/finance/invoice-items/{id}	Delete line item
GET	/finance/invoices/{id}/payments	List payments for invoice
POST	/finance/invoices/{id}/payments	Add payment
GET	/finance/expenses	List expenses; filter by category
POST	/finance/expenses	Log expense
PUT	/finance/expenses/{id}	Update expense
DELETE	/finance/expenses/{id}	Delete expense

Other Modules (Standard CRUD Pattern)

Analytics (/analytics)

Dashboard KPIs, reporting, cross-module aggregations.

Campaigns (/campaigns)

Marketing campaign management and analytics.

Content Calendar (/content-calendar)

Editorial scheduling, post planning.

Messaging (/messaging)

Internal messaging, conversation threads.

Notifications (/notifications)

User notifications, read/unread tracking.

Tasks (/tasks)

Task management, assignment, status tracking.

Support Tickets (/support-tickets)

Client ticket creation, assignment, resolution.

Timesheets (/timesheets)

Employee time tracking, approval workflow.

Meetings (/meetings)

Meeting scheduling, notes, follow-ups.

SEO (/seo)

SEO projects, keyword tracking, reports.

File Manager (/files)

File uploads, document organization.

Users (/users)

User management, profiles, role assignment.



SECTION 8

Security Model



JWT Authentication (Bearer Token)

Access tokens (30 min TTL) signed HS256 HMAC. Refresh tokens (7 days TTL) stored as hashes. Token type claim validated (access vs refresh). Protected routes use `get_current_user` dependency.



Password Security

bcrypt at 12 rounds. Password strength validation on register/change. Constant-time comparison via passlib to prevent timing attacks.



Account Lockout

After 5 consecutive failures → locked 15 minutes. State in `users.locked_until`. Reset on successful login.



Rate Limiting

Fixed-window in-memory by (path, IP). Login: 5/min, Register: 3/min, Refresh: 10/min. Returns HTTP 429 with structured error.



Audit Logging

All auth events → audit_logs table. Fields: action, user_id, IP, device info, old/new data (JSONB), timestamp. Events: register, login_success, login_failed, logout, refresh, access_denied, invalid_token, verification_*, password_reset_*, session_*, new_device_login.



CORS

Specific origins only: localhost:3000/3001, amplivo-front-and-backend.vercel.app, amplivo-2.vercel.app. Credentials allowed.



Security Headers (Every Response)

```
X-Frame-Options: DENY · X-Content-Type-Options: nosniff · Referrer-Policy: strict-origin-when-cross-origin · Permissions-Policy: geolocation=(), microphone=(), camera=() · CSP: default-src 'self'; frame-ancestors 'none'
```



RBAC + Tenant Isolation

Routes protected with `require_roles("role_slug")`. Client portal users scoped to their company via `users.client_id` — enforced by `get_current_client_id` dependency on all relevant endpoints.



Device & Session Tracking

Each login creates session record: device, browser, OS, IP, country, city. Multi-device management: view, terminate individual or all other sessions. Session inactivity timeout (60 min).



SECTION 9

Core Data Models

User Model (users table)

COLUMN	TYPE	DESCRIPTION
id	UUID PK	Auto-generated UUIDv4
email	VARCHAR(255) UNIQUE	Email address (indexed)
username	VARCHAR(50) UNIQUE	Unique username (indexed)
full_name	VARCHAR(150)	Display name
phone	VARCHAR(20) NULL	Optional phone number
hashed_password	VARCHAR(255)	bcrypt hash of password
user_type	TEXT	"internal" "client" etc.
status	TEXT	"pending" "active" "inactive"
role_id	UUID FK → roles	RBAC role assignment
department_id	UUID FK → departments	Org department
branch_id	UUID FK → branches	Office branch
reporting_manager_id	UUID FK → users	Hierarchical reporting
client_id	UUID FK → clients	Portal user's company link
is_active	BOOLEAN	Account active status
is_verified	BOOLEAN	Email verification status
verified_at	TIMESTAMPTZ NULL	Email verification timestamp
last_login_at	TIMESTAMPTZ NULL	Last successful login
last_login_ip	VARCHAR(45) NULL	IPv4/IPv6 of last login
last_seen	TIMESTAMPTZ NULL	Last API activity (activity middleware)
failed_login_attempts	INTEGER	Consecutive failure count
locked_until	TIMESTAMPTZ NULL	Account lockout expiry
is_deleted	BOOLEAN	Soft-delete flag (indexed)
deleted_at / deleted_by	TIMESTAMPTZ / UUID	Soft-delete audit trail
created_at / updated_at	TIMESTAMPTZ	Server-set timestamps

RefreshToken Model (refresh_tokens)

COLUMN	TYPE	DESCRIPTION
id	UUID PK	Token ID
user_id	UUID FK → users CASCADE	Owning user
token_hash	VARCHAR(255) UNIQUE	Hash of the raw token
is_revoked	BOOLEAN	Revocation status
expires_at	TIMESTAMPTZ	Token expiry
device_name / browser / os	VARCHAR	Device fingerprint at issue time
ip_address	VARCHAR(45)	IP at issue time
last_used	TIMESTAMPTZ NULL	Last use timestamp
revoked_at	TIMESTAMPTZ NULL	Revocation timestamp

AuditLog Model (audit_logs)

COLUMN	TYPE	DESCRIPTION
id	UUID PK	Log entry ID
table_name	VARCHAR(100) NULL	Affected table
record_id	UUID NULL	Affected record ID
action	VARCHAR(50)	AuditAction enum value
old_data / new_data	JSONB NULL	State snapshots before/after
performed_by	UUID FK → users NULL	Actor user ID
ip_address	VARCHAR(45) NULL	Client IP address
created_at	TIMESTAMP	Event timestamp (server)



SECTION 10

Frontend Technology Details

Application Structure

```
frontend/src/
├─ app/                ← Next.js App Router pages
│  ├─ admin/           ← Admin panel
│  ├─ crm/             ← CRM dashboard
│  ├─ employee/        ← Employee portal
│  ├─ hr/              ← HR module
│  ├─ portal/          ← Client portal (messages, etc.)
│  ├─ sales/           ← Sales module
│  ├─ login/ register/ ← Auth pages
│  └─ (public pages)   ← about, blog, careers, contact, faq...
├─ components/         ← Reusable UI components
├─ services/           ← API client (axios)
├─ store/              ← Zustand state stores
├─ hooks/              ← Custom React hooks
├─ types/              ← TypeScript type definitions
├─ lib/                ← Utility functions
└─ constants/          ← App constants
```

API Client (src/services/api.ts)

- **Base URL:** `NEXT_PUBLIC_API_URL` or `http://127.0.0.1:8000/api/v1`
- **Request Interceptor:** Attaches `Authorization: Bearer <token>` from Zustand auth store
- **401 Handling:** Silently refreshes token via `POST /auth/refresh` ; queues concurrent 401 requests; falls back to logout on refresh failure

Frontend Dependencies

PACKAGE	VERSION	PURPOSE
next	^16.2.10	React framework, App Router, SSR/SSG
react / react-dom	^19.2.4	UI library with Server Components
typescript	5.9.3	Type-safe JavaScript
tailwindcss	^4	Utility-first CSS framework
axios	^1.18.1	HTTP client with interceptors
zustand	^5.0.14	Lightweight global state
react-hook-form	^7.82.0	Performant form state
@hookform/resolvers	^5.4.0	Zod integration for RHF
zod	^4.4.3	TypeScript-first schema validation

PACKAGE	VERSION	PURPOSE
recharts	^3.9.2	Composable chart library
lucide-react	^0.469.0	Open-source icon set
react-icons	^5.7.0	Popular icon packs as components
clsx + tailwind-merge	^2 / ^3	Conditional CSS class utilities



SECTION 11

Environment Variables Reference

⚠ Security: Never commit `.env` files to version control. Use Render environment groups and Vercel project settings for production secrets.

Backend (Backend/.env)

VARIABLE	REQUIRED	DEFAULT / NOTES
<code>DATABASE_URL</code>	Required	postgresql+asyncpg://user:pass@host:port/dbname
<code>JWT_SECRET_KEY</code>	Required	Long random string — auto-generated by Render in prod
<code>ENVIRONMENT</code>	Optional	development production (default: development)
<code>DB_POOL_SIZE</code>	Optional	5 (dev) / 3 (prod)
<code>DB_MAX_OVERFLOW</code>	Optional	10 (dev) / 5 (prod)
<code>DB_POOL_RECYCLE_SECONDS</code>	Optional	1800 (dev) / 300 (prod)
<code>DB_POOL_TIMEOUT_SECONDS</code>	Optional	30 (dev) / 60 (prod)
<code>DB_STARTUP_TIMEOUT_SECONDS</code>	Optional	5.0 (dev) / 10.0 (prod)
<code>DB_SSL_MODE</code>	Optional	require (remote) / disable (local)
<code>DB_ECHO</code>	Optional	false (dev only: true for SQL logging)
<code>SUPABASE_URL</code>	Optional	For Supabase SDK features (not used by SQLAlchemy)
<code>SUPABASE_ANON_KEY</code>	Optional	Public anon key
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Optional	Admin operations key
<code>JWT_ALGORITHM</code>	Optional	HS256
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Optional	30
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	Optional	7
<code>BCRYPT_ROUNDS</code>	Optional	12
<code>CORS_ORIGINS</code>	Optional	Comma-separated or JSON array of allowed origins
<code>MAX_FAILED_LOGIN_ATTEMPTS</code>	Optional	5
<code>ACCOUNT_LOCK_MINUTES</code>	Optional	15

VARIABLE	REQUIRED	DEFAULT / NOTES
RATE_LIMIT_LOGIN_PER_MINUTE	Optional	5 (dev) / 20 (prod)
RATE_LIMIT_REGISTER_PER_MINUTE	Optional	3 (dev) / 10 (prod)
RATE_LIMIT_REFRESH_PER_MINUTE	Optional	10 (dev) / 30 (prod)
EMAIL_VERIFICATION_TOKEN_EXPIRE_HOURS	Optional	24
PASSWORD_RESET_TOKEN_EXPIRE_MINUTES	Optional	30
SESSION_INACTIVITY_TIMEOUT_MINUTES	Optional	60
REDIS_URL	Optional	For future distributed rate limiting

Frontend (frontend/.env.local)

VARIABLE	REQUIRED	VALUE
NEXT_PUBLIC_API_URL	Required	https://amplivo-front-and-backend.onrender.com/api/v1



SECTION 12

Deployment & Infrastructure

Backend — Render Web Service

- **Runtime:** Python 3.12
- **Root Dir:** Backend/
- **Build:** `pip install -r requirements.txt && alembic upgrade head`
- **Start:** `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
- **URL:** amplivo-front-and-backend.onrender.com

Frontend — Vercel

- **Runtime:** Node.js 20+
- **Root Dir:** frontend/
- **Build:** `npm ci && npm run build`
- **Start:** `npm start`
- **URL:** amplivo-front-and-backend.vercel.app

Database — Supabase PostgreSQL

- **Region:** ap-south-1 (Mumbai, India)
- **Pooler:** Supavisor, port 6543, transaction mode
- **SSL:** Required (cert verification disabled for pooler)
- **Migrations:** Alembic — applied at each deploy via build command
- **JWT Secret:** Auto-generated by Render on first deploy (`generateValue: true`)

Local Development

Backend Local

```
cd Backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
# Set DB_SSL_MODE=disable in .env
alembic upgrade head
uvicorn app.main:app --reload
# API: http://localhost:8000
```

Frontend Local

```
cd frontend
npm install
# Create frontend/.env.local:
#
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
npm run dev
# App: http://localhost:3000
```



Health & Monitoring

Health Check Endpoints

METHOD	PATH	USE CASE	RESPONSE
GET	/health	Liveness probe — never touches DB	{"status": "ok"} HTTP 200
GET	/health/database	Readiness probe — executes SELECT 1	{"status": "healthy", "database": "connected", "latency": "2.5ms"} 200 or 503

 **Probe Strategy:** Use `/health` as the liveness probe (container restart decisions) and `/health/database` as the readiness probe (traffic routing). This prevents unnecessary restarts when DB is temporarily unreachable during deployment.

Live API Documentation

TOOL	URL	DESCRIPTION
Swagger UI	/api/v1/docs	Interactive try-it-out API explorer
ReDoc	/api/v1/redoc	Clean, readable API reference
OpenAPI JSON	/api/v1/openapi.json	Machine-readable OpenAPI 3.0 spec

Startup & Shutdown Behavior

- Startup: DB connectivity check bounded to `DB_STARTUP_TIMEOUT_SECONDS` — app still boots even if DB unreachable
- Startup failure: logs warning, continues boot — `/health` returns ok, `/health/database` returns 503
- Shutdown: `engine.dispose()` called via lifespan context — graceful connection pool cleanup
- Static files: `/uploads` served from `Backend/uploads/` via Starlette StaticFiles